Email Spam Classifier | SMS Spam Classifier

Este projeto tem como objetivo aplicar técnicas de análise de dados e machine learning para resolver problemas reais, identificando padrões e tendências nos dados. Utilizando ferramentas como Python, Pandas, Scikit-learn e Power BI, o projeto envolve etapas de limpeza de dados, feature engineering e treinamento de modelos preditivos, além da apresentação de resultados visuais interativos.

Inspiração do projeto: https://www.youtube.com/watch?v=YncZ0WwxyzU&list=PLKnIA16_RmvY5eP91BGPa0vXUYmIdtfPQ&index=3

Data-set: https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset

1. Importando as bibliotecas que preciso:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from google.colab import files
```

2. Carregando o data-set:

```
# Instalar o kaggle
!pip install kaggle -q
```

```
# Colocar sua API Key do Kaggle:
files.upload()
```

```
Escolher arquivos kaggle.json
• kaggle.json(application/json) - 69 bytes, last modified: 12/01/2025 - 100% done
Saving kaggle.json to kaggle.json
{'kaggle.json': b'{"username":"educoqueiro11","key":"efb78b467f3a8c686837c78e44d85e9b"}'}
```

```
# Criando uma pasta para colocar os arquivos:
!mkdir -p ~/.kaggle
```

```
# Copiando paa a pasta que foi criada
!cp kaggle.json ~/.kaggle
```

```
# Atribuindo uma conexão na pasta:
!chmod 600 ~/.kaggle/kaggle.json
```

```
# Listando os data sets disponíveis:
!kaggle datasets list
```

```
ref                                                            title                                          size  lastUpdated
-------------------------------------------------------------  ---------------------------------------------  ----  -------------
anandshaw2001/netflix-movies-and-tv-shows                      Netflix Movies and TV Shows                     1MB  2025-01-03 10
stealthtechnologies/predict-student-performance-dataset        Predict Student Performance                    12KB  2024-12-26 12
ankushpanday1/heart-attack-in-youth-vs-adult-in-germany        Heart Attack in Youth Vs Adult in Germany        6MB  2025-01-08 14
taweilo/wine-quality-dataset-balanced-classification           Wine Quality dataset - Classification          377KB  2025-01-07 04
prajwaldongre/global-financial-giants-by-revenue-2024          Global Financial Giants by Revenue 2024         2KB  2025-01-08 07
yamaerenay/spotify-dataset-1921-2020-160k-tracks               Spotify Dataset 1921-2020, 160k+ Tracks        16MB  2025-01-06 21
oktayrdeki/heart-disease                                       Heart Disease                                  568KB  2024-12-29 13
sebastianwillmann/beverage-sales                               Beverage Sales                                 119MB  2025-01-02 21
stealthtechnologies/gdp-growth-of-african-countries            GDP Growth of African Countries                 13KB  2025-01-02 06
ankushpanday1/heart-attack-in-youth-vs-adult-in-france         Heart Attack in Youth VS Adult in France         8MB  2025-01-07 17
anandshaw2001/mobile-apps-screentime-analysis                  Mobile Apps ScreenTime Analysis                  2KB  2024-12-31 18
umerhaddii/apple-stock-data-2025                               Apple Stock Data 2025                          301KB  2025-01-03 16
govindaramsriram/energy-consumption-dataset-linear-regression  Energy Consumption Dataset - Linear Regression  16KB  2025-01-06 16
bhadramohit/customer-shopping-latest-trends-dataset            Customer Shopping (Latest Trends) Dataset       76KB  2024-11-23 15
anandshaw2001/imdb-data                                        Full IMDb Movies Dataset                       153MB  2025-01-03 05
ankushpanday1/diabetes-in-youth-vs-adult-in-india              Diabetes in Youth Vs Adult in India              3MB  2025-01-04 07
ankushpanday1/heart-attack-in-youth-of-india                   Heart attack in youth of India                 298KB  2025-01-02 15
ankushpanday1/heart-attack-in-china-youth-vs-adult             Heart Attack in China Youth Vs Adult            34MB  2025-01-06 14
simronw/tesla-stock-data-2024                                  TESLA Stock Data 2024                           91KB  2024-12-14 06
ankitrajmishra/walmart                                         walmart                                        236KB  2024-12-31 19
```

```
# Baixando o data-set:
```

```
!kaggle datasets download uciml/sms-spam-collection-dataset
```

```
Dataset URL: https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset
License(s): unknown
Downloading sms-spam-collection-dataset.zip to /content
  0% 0.00/211k [00:00<?, ?B/s]
100% 211k/211k [00:00<00:00, 81.1MB/s]
```

```
# Desconectando o arquivo:
!unzip /content/sms-spam-collection-dataset.zip
```

```
Archive:  /content/sms-spam-collection-dataset.zip
  inflating: spam.csv
```

```
# Carregando o data-set:

df = pd.read_csv('/content/spam.csv', encoding='ISO-8859-1')
```

```
df.head()
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

Próximas etapas: Gerar código com df | Ver gráficos recomendados | New interactive sheet

```
df.shape
```

```
(5572, 5)
```

3. Data Cleaning:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   v1          5572 non-null   object
 1   v2          5572 non-null   object
 2   Unnamed: 2  50 non-null     object
 3   Unnamed: 3  12 non-null     object
 4   Unnamed: 4  6 non-null      object
dtypes: object(5)
memory usage: 217.8+ KB
```

Vemos que v1 e v2 são as colunas importantes, o resto pode simplesmente ser apagado!

```
# Apagrando as últimas 3 colunas:

df.drop(columns = ['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], inplace = True)
```

```
# Esse é o nosso data-set:
df.sample(5)
```

	v1	v2
5195	ham	Darren was saying dat if u meeting da ge den w...
563	spam	GENT! We are trying to contact you. Last weeke...
1871	ham	Dont know supports ass and srt i thnk. I think...
3476	ham	I got it before the new year cos yetunde said ...
1144	ham	Really... I tot ur paper ended long ago... But...

```
# Renomeando as colunas restantes:

df.rename(columns = {'v1':'target','v2':'text'}, inplace = True)
df.sample(5)
```

	target	text
1523	ham	Yup ok thanx...
2736	ham	Really? I crashed out cuddled on my sofa.
3609	ham	Call me. I m unable to cal. Lets meet bhaskar,...
4028	ham	[□Û_] anyway, many good evenings to u! s
647	spam	PRIVATE! Your 2003 Account Statement for shows...

```
# Convertendo os valores categóricos em numéricos (LabelEncoder):

from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()

df['target'] = encoder.fit_transform(df['target'])
df.head(5)
```

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

Próximas etapas: Gerar código com df | Ver gráficos recomendados | New interactive sheet

```
# Valores faltando:

df.isnull().sum()
```

	0
target	0
text	0

dtype: int64

Vemos que não há valores faltando, então podemos seguir.

```
# Conferindo se temos valores duplicados:

df.duplicated().sum()
```

```
403
```

Vemos que existem 403 valores duplicados, então vamos remover eles.

```
# Removendo os valores duplicados:

df = df.drop_duplicates(keep = 'first')

df.duplicated().sum()
```

```
0
```

```
df.shape
```

```
(5169, 2)
```

4. EDA (Exploratory Data Analysis - Análise Exploratória de Dados)

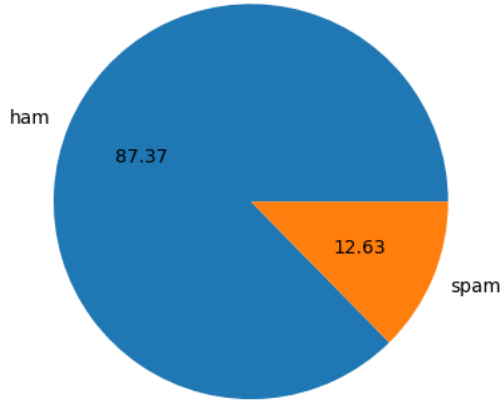
```
df['target'].value_counts()
```

target	count
0	4516
1	653

dtype: int64

Vemos que há uma certa difereça entre os conjuntos de dados em valores.

```
import matplotlib.pyplot as plt

plt.pie(df['target'].value_counts(), labels = ['ham', 'spam'], autopct ='%0.2f')
plt.show()
```



Podemos ver que os dados estão desbalanceados.

5. Data Engineering:

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.10/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.10/dist-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (from nltk) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.10/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from nltk) (4.67.1)
```

```
import nltk
from nltk.tokenize import word_tokenize
```

```
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

```
# Número de caracteres usados em cada uma:
df['text'].apply(len)
```

	text
0	111
1	29
2	155
3	49
4	61
...	...
5567	161
5568	37
5569	57
5570	125
5571	26

5169 rows × 1 columns

dtype: int64

```
df['num_characters'] = df['text'].apply(len)
```

```
df.head()
```

	target	text	num_characters
0	0	Go until jurong point, crazy.. Available only ...	111
1	0	Ok lar... Joking wif u oni...	29
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155
3	0	U dun say so early hor... U c already then say...	49
4	0	Nah I don't think he goes to usf, he lives aro...	61

Próximas etapas: Gerar código com df | Ver gráficos recomendados | New interactive sheet

```
# Número de palavras:
import nltk
nltk.download('punkt_tab')
# Vamos tokenizar a variável texto, fazendo cada palavra um elemento:
df['num_words'] = df['text'].apply(lambda x: len(word_tokenize(x)))
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

```
df.head()
```

	target	text	num_characters	num_words
0	0	Go until jurong point, crazy.. Available only ...	111	24
1	0	Ok lar... Joking wif u oni...	29	8
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37
3	0	U dun say so early hor... U c already then say...	49	13
4	0	Nah I don't think he goes to usf, he lives aro...	61	15

Próximas etapas: Gerar código com df | Ver gráficos recomendados | New interactive sheet

```
# número de sentenças:

df['num_sentences'] = df['text'].apply(lambda x:len (nltk.sent_tokenize(x)))

df.head()
```

	target	text	num_characters	num_words	num_senteces	num_sentences
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	2
1	0	Ok lar... Joking wif u oni...	29	8	2	2
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	2
3	0	U dun say so early hor... U c already then say...	49	13	1	1
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	1

Próximas etapas: Gerar código com df | Ver gráficos recomendados | New interactive sheet

```
df[['num_characters', 'num_words', 'num_sentences']].describe()
```

	num_characters	num_words	num_sentences
count	5169.000000	5169.000000	5169.000000
mean	78.977945	18.455794	1.965564
std	58.236293	13.324758	1.448541
min	2.000000	1.000000	1.000000
25%	36.000000	9.000000	1.000000
50%	60.000000	15.000000	1.000000
75%	117.000000	26.000000	2.000000
max	910.000000	220.000000	38.000000

```
# Apenas os ham values:
df[df['target'] == 0][['num_characters', 'num_words', 'num_sentences']].describe()
```

	num_characters	num_words	num_sentences
count	4516.000000	4516.000000	4516.000000
mean	70.459256	17.123782	1.820195
std	56.358207	13.493970	1.383657
min	2.000000	1.000000	1.000000
25%	34.000000	8.000000	1.000000
50%	52.000000	13.000000	1.000000
75%	90.000000	22.000000	2.000000
max	910.000000	220.000000	38.000000

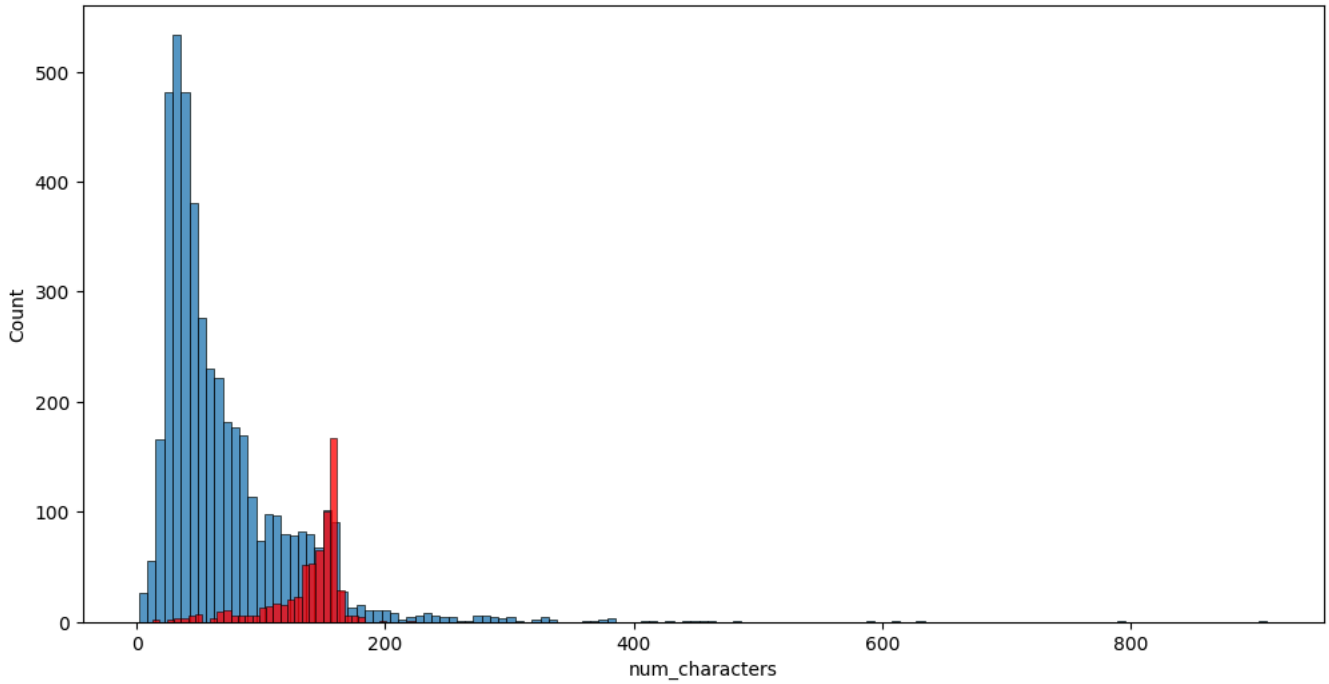
```
# Apenas os spam values:
df[df['target'] == 1][['num_characters', 'num_words', 'num_sentences']].describe()
```

	num_characters	num_words	num_sentences
count	653.000000	653.000000	653.000000
mean	137.891271	27.667688	2.970904
std	30.137753	7.008418	1.488425
min	13.000000	2.000000	1.000000
25%	132.000000	25.000000	2.000000
50%	149.000000	29.000000	3.000000
75%	157.000000	32.000000	4.000000
max	224.000000	46.000000	9.000000

Vemos que no geral as métricas dos emails de spam são maiores que os que não são!

```
import seaborn as sns

plt.figure(figsize = (12, 6))
sns.histplot(df[df['target'] == 0]['num_characters'])
sns.histplot(df[df['target'] == 1]['num_characters'], color = 'red')
```

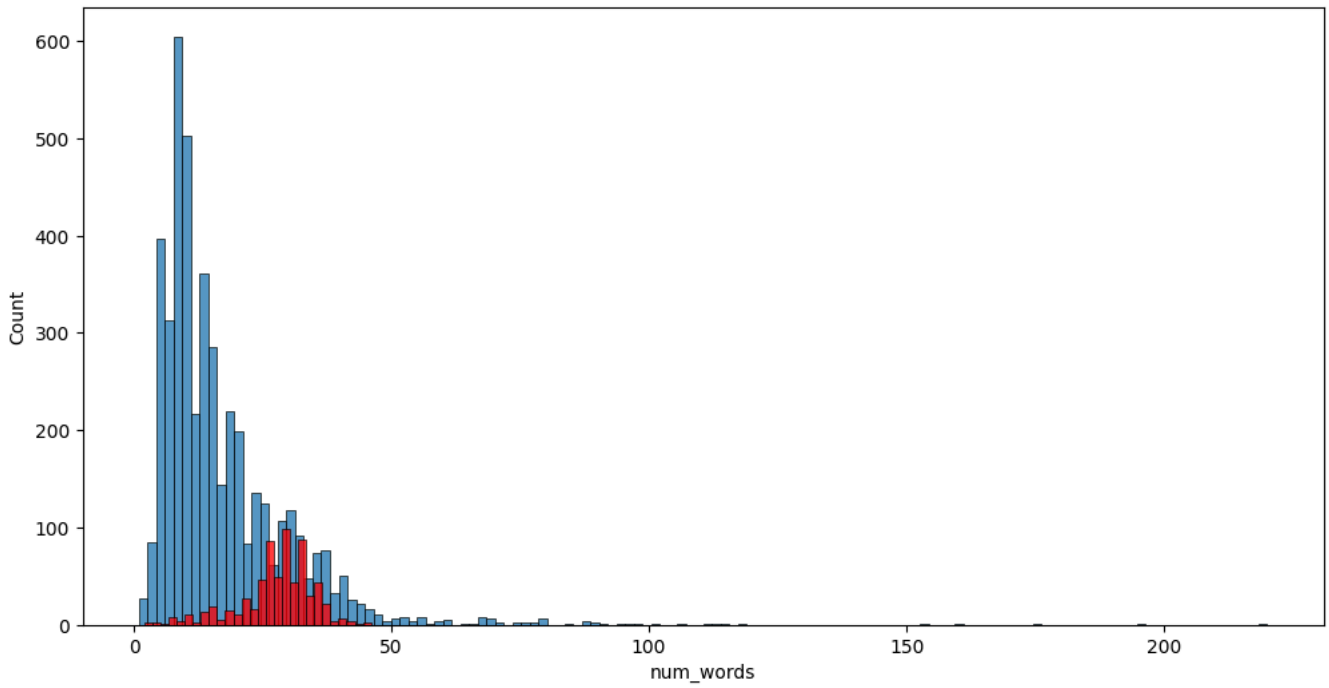
```
<Axes: xlabel='num_characters', ylabel='Count'>
```



Podemos notar que as ham mensagens tem muitos menos caracteres que os spam!

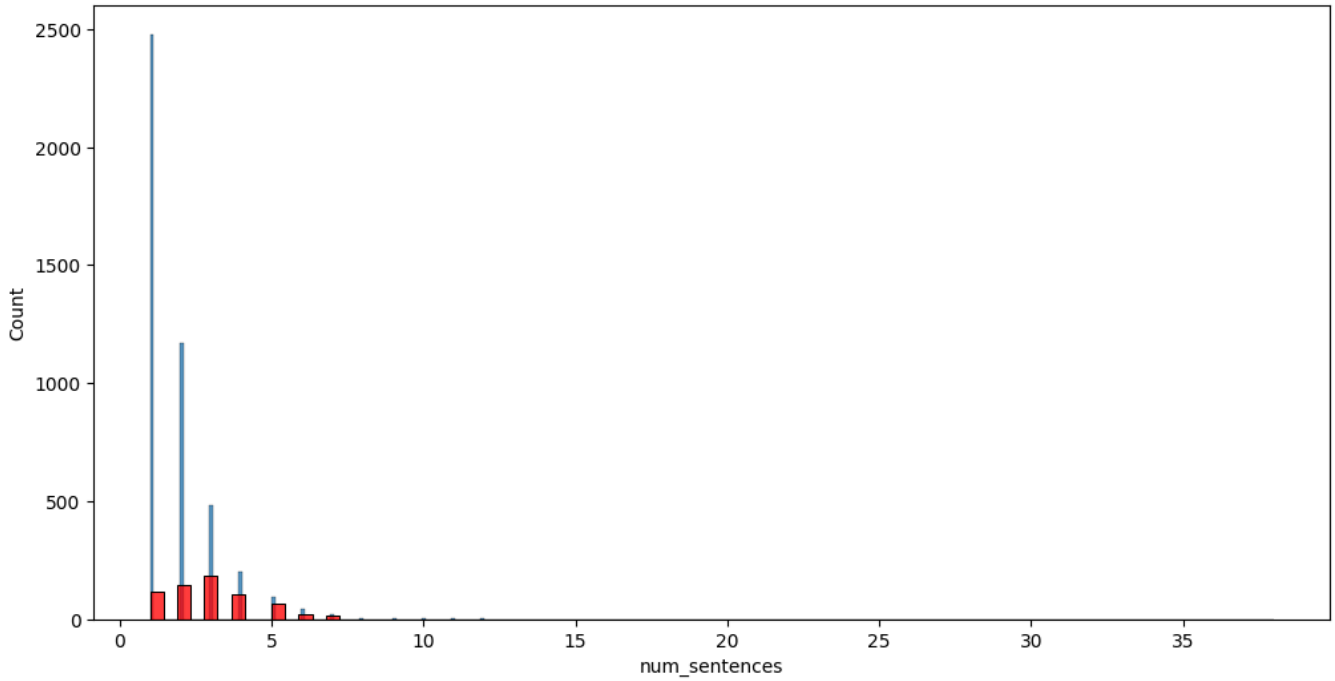
```
# Fazendo o mesmo para as outras variáveis criadas:


plt.figure(figsize = (12, 6))
sns.histplot(df[df['target'] == 0]['num_words'])
sns.histplot(df[df['target'] == 1]['num_words'], color = 'red')
```

```
<Axes: xlabel='num_words', ylabel='Count'>
```



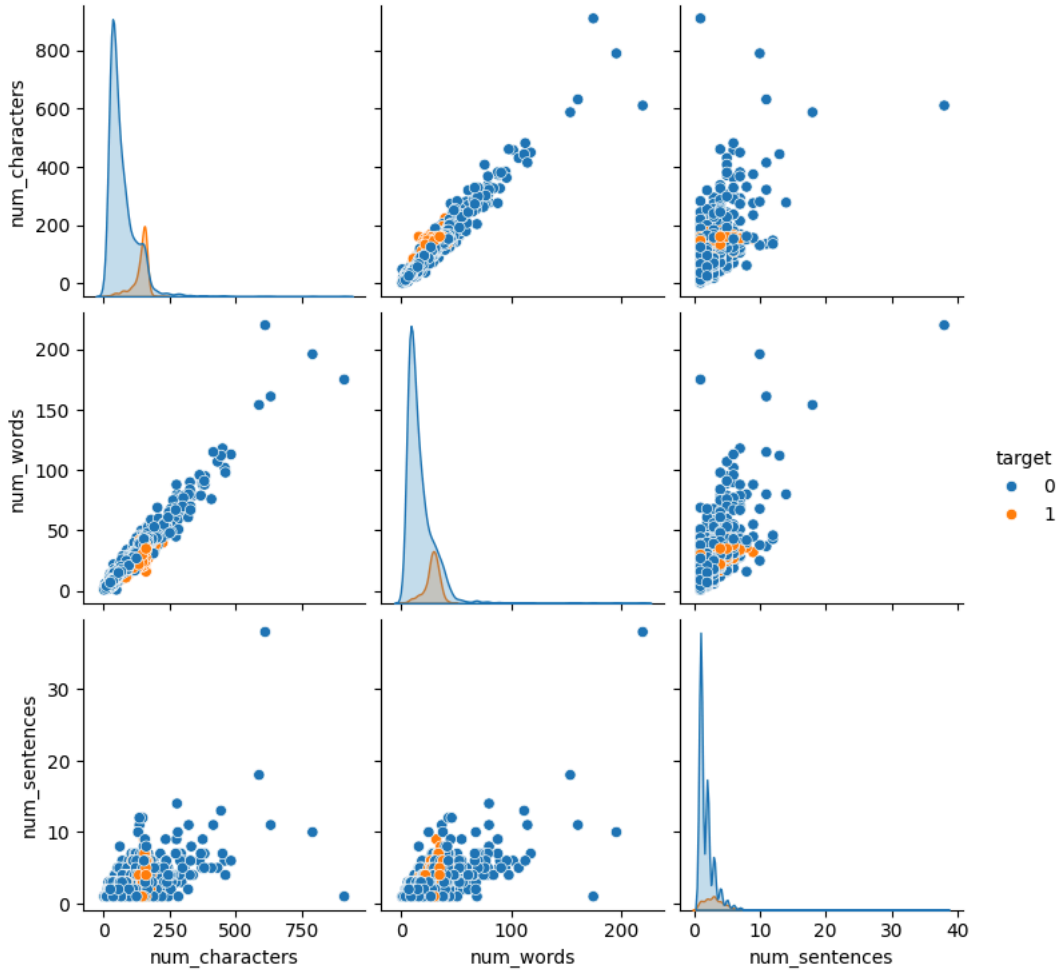

```
# Fazendo o mesmo para as outras variáveis criadas:


plt.figure(figsize = (12, 6))
sns.histplot(df[df['target'] == 0]['num_sentences'])
sns.histplot(df[df['target'] == 1]['num_sentences'], color = 'red')
```

```
<Axes: xlabel='num_sentences', ylabel='Count'>
```




```
# Vamos ver a correlação dos valores:

sns.pairplot(df, hue = 'target')
```

```
<seaborn.axisgrid.PairGrid at 0x7fb321ca59c0>
```



O gráfico de pares mostra a relação entre o número de caracteres, palavras e sentenças em mensagens, comparando-as com a variável target (0 ou 1). Há uma tendência clara de que mensagens mais longas (em termos de caracteres e palavras) sejam associadas ao target 1 (provavelmente spam). A relação linear entre o número de palavras e sentenças sugere que mensagens maiores têm mais sentenças.

```
df.info()
```

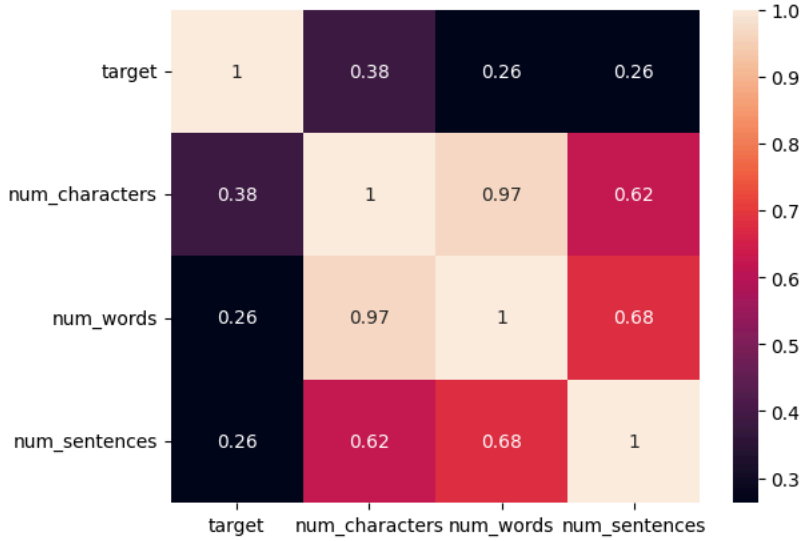
```
<class 'pandas.core.frame.DataFrame'>
Index: 5169 entries, 0 to 5571
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   target          5169 non-null   int64 
 1   text            5169 non-null   object
 2   num_characters  5169 non-null   int64 
 3   num_words       5169 non-null   int64 
 4   num_senteces    5169 non-null   int64 
 5   num_sentences   5169 non-null   int64 
dtypes: int64(5), object(1)
memory usage: 282.7+ KB
```

```
df.drop(columns=['text']).corr()
```

	target	num_characters	num_words	num_sentences
target	1.000000	0.384717	0.262912	0.263939
num_characters	0.384717	1.000000	0.965760	0.624139
num_words	0.262912	0.965760	1.000000	0.679971
num_sentences	0.263939	0.624139	0.679971	1.000000

```
sns.heatmap(df.drop(columns=['text']).corr(), annot=True)
```

```
<Axes: >
```



6. Data Preprocessing:

- Letras minúsculas
- Tokenização
- Remoção de caracteres especiais
- Remoção de stop words e pontuação
- Stemming (redução para a raiz das palavras)

```
df.head()
```

	target	text	num_characters	num_words	num_sentences
0	0	Go until jurong point, crazy.. Available only ...	111	24	2
1	0	Ok lar... Joking wif u oni...	29	8	2
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2
3	0	U dun say so early hor... U c already then say...	49	13	1
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1

Próximas etapas: Gerar código com df | Ver gráficos recomendados | New interactive sheet

```
# Criando uma função para passar pelas frases e conferter tudo para letra minuscula:

def transform_text(text):
  text = text.lower()
  return text


transform_text('Hi how are You?')
```

```
'hi how are you?'
```

Vemos que foi tudo para minúscula como gostariamos!

```
# Tokenização:

def transform_text(text):
  text = text.lower()
  text = nltk.word_tokenize(text)
  return text


transform_text('Hi how are You?')
```

```
['hi', 'how', 'are', 'you', '?']
```

Vemos que todas as palavras foram separadas!

```
# Removendo caracteres especiais:

def transform_text(text):
  text = text.lower()
  text = nltk.word_tokenize(text)

  y = []

  for i in text:
    if i.isalnum():
      y.append(i)

  return y


transform_text('Hi !o! are%%% You?.')
```

```
['hi', 'o', 'are', 'you']
```

Vemos que ele não puxa os caracteres especiais!

Stop Words são palavras muito comuns em um idioma que, em geral, não agregam valor semântico relevante em tarefas de análise de texto, como modelos de Machine Learning ou Processamento de Linguagem Natural (NLP).

```
# Removendo stop words:

from nltk.corpus import stopwords
nltk.download('stopwords')
stopwords.words('english')
```

```
 y ,
'ain',
'aren',
"aren't",
'couldn',
"couldn't",
'didn',
"didn't",
'doesn',
"doesn't",
'hadn',
"hadn't",
'hasn',
"hasn't",
'haven',
"haven't",
'isn',
"isn't",
'ma',
'mightn',
"mightn't",
'mustn',
"mustn't",
'needn',
"needn't",
'shan',
"shan't",
'shouldn',
"shouldn't",
'wasn',
"wasn't",
'weren',
"weren't",
'won',
"won't",
'wouldn',
"wouldn't"]
```

```
# Vendo algumas em pt:
stopwords.words('portuguese')
```

```
        'tivessemos',
        'tu',
        'tua',
        'tuas',
        'um',
        'uma',
        'você',
        'vocês',
        'vos']
```

```
import string
string.punctuation
```

```
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

```
# Removendo stop words e pontuações:

def transform_text(text):
  text = text.lower()
  text = nltk.word_tokenize(text)

  y = []

  for i in text:
    if i.isalnum():
      y.append(i)

  text = y[:]
  y.clear()
  for i in text:
    if i not in stopwords.words('english') and i not in string.punctuation:
      y.append(i)

  return y
```

```
transform_text("Hi !o! are%%% You? It's a beautiful day here in Brazil")
```

```
['hi', 'beautiful', 'day', 'brazil']
```

O Stemming é uma técnica de Processamento de Linguagem Natural (NLP) que consiste em reduzir palavras a sua raiz (stem), removendo prefixos e sufixos. Essa raiz geralmente não é uma palavra válida, mas representa a base semântica da palavra.

```
#Stemming:

from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()
ps.stem('loving')
```

```
'love'
```

```
def transform_text(text):
  text = text.lower()
  text = nltk.word_tokenize(text)

  y = []

  for i in text:
    if i.isalnum():
      y.append(i)

  text = y[:]
  y.clear()
  for i in text:
    if i not in stopwords.words('english') and i not in string.punctuation:
      y.append(i)

  text = y[:]
  y.clear()

  for i in text:
    y.append(ps.stem(i))

  return " ".join(y)
```

```
transform_text('I loved the Youtube Lectures on Machine Learning... How about you?')
```

```
'love youtub lectur machin learn'
```

```
# Aplicando a função no nosso data-set:

df['transformed_text'] = df['text'].apply(transform_text)
df.head()
```

	target	text	num_characters	num_words	num_sentences	transformed_text
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong point crazi avail bugi n great world...
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 wkli comp win fa cup final tkt 21...
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli hor u c alreadi say

Próximas etapas: Gerar código com df | Ver gráficos recomendados | New interactive sheet

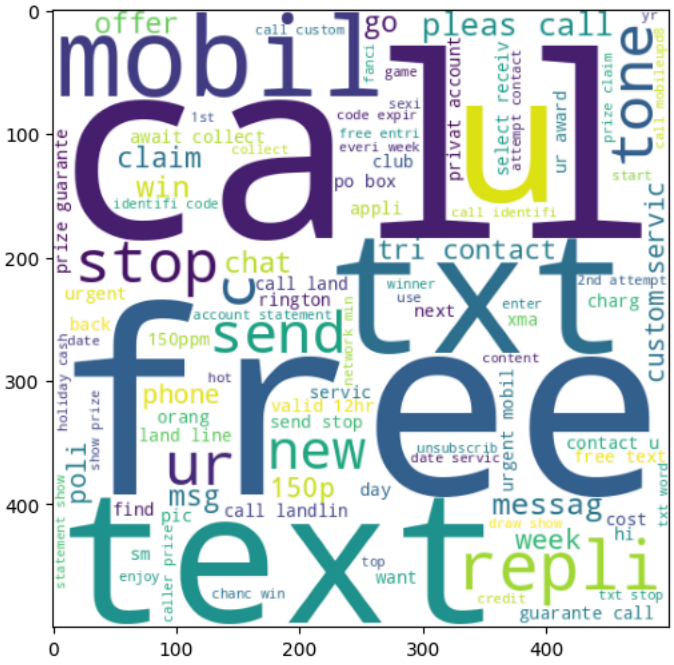
Vemos que estão aplicados todos os passos da função! Vamos ver agora as palavras mais recorrentes!

```
from wordcloud import WordCloud
wc = WordCloud(width = 500, height = 500, min_font_size = 10, background_color = 'white')
```

```
spam_wc = wc.generate(df[df['target'] == 1]['transformed_text'].str.cat(sep = " "))
```

```
plt.figure(figsize = (16,6))
plt.imshow(spam_wc)
```

```
<matplotlib.image.AxesImage at 0x7fb321496740>
```

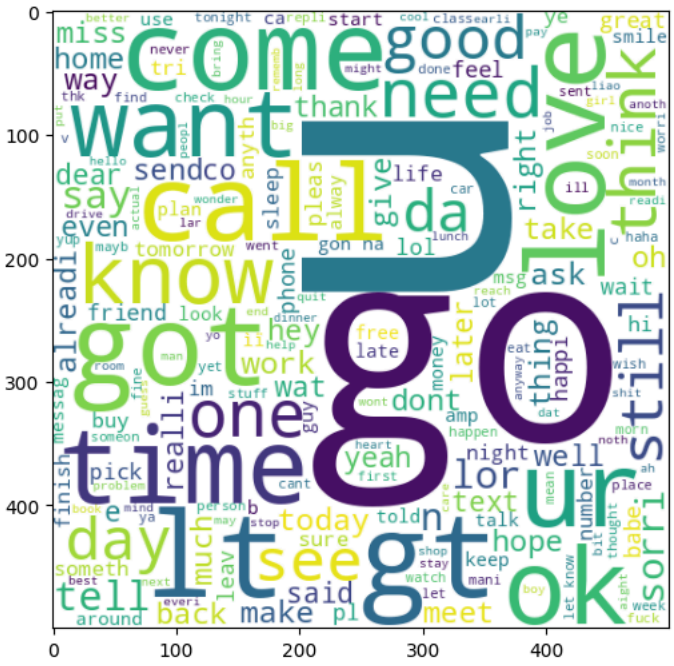


O Word Cloud revela palavras comuns em mensagens de spam, destacando termos como "call" (ligar), "free" (grátis), "win" (ganhar) e "txt" (mensagem). Essas palavras indicam que mensagens de spam frequentemente oferecem prêmios falsos, promoções ou tentam induzir o usuário a entrar em contato por meio de ligações ou SMS. Termos como "stop" também são comuns, sugerindo instruções para cancelar o spam. O foco dessas mensagens é enganação e urgência, explorando ofertas e solicitações rápidas.

```
ham_wc = wc.generate(df[df['target'] == 0]['transformed_text'].str.cat(sep = " "))
```

```
plt.figure(figsize = (16,6))
plt.imshow(ham_wc)
```

```
<matplotlib.image.AxesImage at 0x7fb32104cc10>
```



O Word Cloud de mensagens ham (não spam) mostra palavras comuns em comunicações cotidianas. Os termos mais frequentes incluem "go", "come", "got", "time", "call" e "want", indicando conversas relacionadas a planejamentos, encontros e comunicação pessoal. Palavras como "good", "love" e "ok" mostram um tom amigável e positivo. Diferente do spam, essas mensagens têm um tom informal e natural, focado em relacionamentos, interações pessoais e necessidades diárias.

```
# Vamos converter os valores de txt para uma lista:

spam_corpus = []
for msg in df[df['target'] == 1]['transformed_text'].tolist():
  for word in msg.split():
      spam_corpus.append(word)
```

```
len(spam_corpus)
```

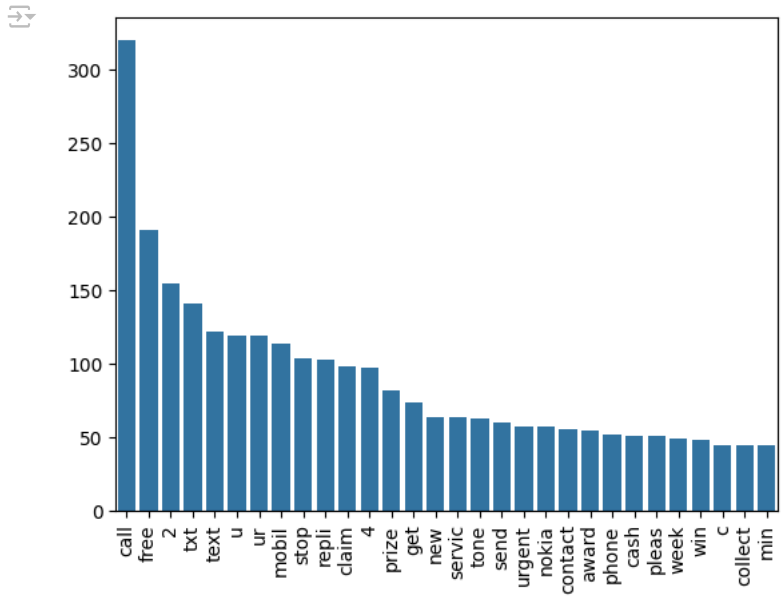
```
9939
```

```
from collections import Counter
Counter(spam_corpus).most_common(30)
```

```
[('call', 320),
 ('free', 191),
 ('2', 155),
 ('txt', 141),
 ('text', 122),
 ('u', 119),
 ('ur', 119),
 ('mobil', 114),
 ('stop', 104),
 ('repli', 103),
 ('claim', 98),
 ('4', 97),
 ('prize', 82),
 ('get', 74),
 ('new', 64),
 ('servic', 64),
 ('tone', 63),
 ('send', 60),
 ('urgent', 57),
 ('nokia', 57),
 ('contact', 56),
 ('award', 55),
 ('phone', 52),
 ('cash', 51),
 ('pleas', 51),
 ('week', 49),
 ('win', 48),
 ('c', 45),
 ('collect', 45),
 ('min', 45)]
```

Vemos que corresponde com o gráfico de núvem das palavras visto acima!

```
from collections import Counter
sns.barplot(x=pd.DataFrame(Counter(spam_corpus).most_common(30))[0], y=pd.DataFrame(Counter(spam_corpus).most_common(30))[1])
plt.xticks(rotation='vertical')
plt.xlabel('')
plt.ylabel('')
plt.show()
```



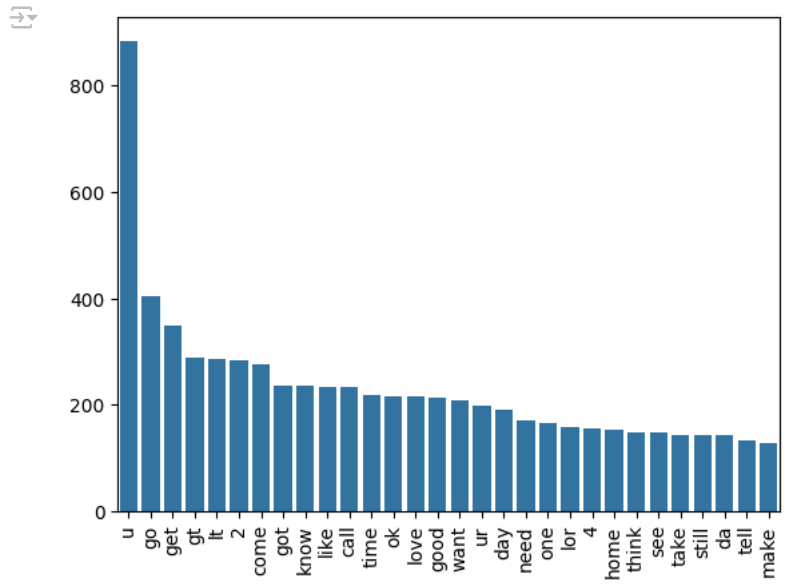
O gráfico de barras mostra as 30 palavras mais frequentes em mensagens spam. As palavras "call", "free", "txt" e "claim" são as mais recorrentes, indicando que essas mensagens costumam conter ofertas gratuitas, solicitações de ligação ou prêmios. Termos como "win", "prize" e "urgent" sugerem tentativas de criar um senso de urgência ou engano. O padrão observado é comum em mensagens promocionais falsas, que buscam atrair atenção e enganar o destinatário.

```
# Vamos converter os valores de txt para uma lista (agora para ham values):
ham_corpus = []
for msg in df[df['target'] == 0]['transformed_text'].tolist():
  for word in msg.split():
      ham_corpus.append(word)
```

```
len(ham_corpus)
```

```
35404
```

```
from collections import Counter
sns.barplot(x=pd.DataFrame(Counter(ham_corpus).most_common(30))[0], y=pd.DataFrame(Counter(ham_corpus).most_common(30))[1])
plt.xticks(rotation='vertical')
plt.xlabel('')
plt.ylabel('')
plt.show()
```



O gráfico de barras mostra as 30 palavras mais frequentes em mensagens ham (não spam). A palavra "u" é a mais comum, destacando um tom informal e pessoal nas mensagens. Termos como "go", "get", "come", "call" e "time" sugerem planejamentos e interações sociais diárias. Além disso, palavras como "love" e "good" indicam um tom mais positivo e amigável, característico de conversas pessoais. Essas mensagens tendem a focar em relacionamentos e tarefas cotidianas, diferentemente de mensagens spam.

7. Model building:

Primeiramente vamos converter o nosso texto em matrizes numéricas para que o modelo possa entender:

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer()
```

```
X = cv.fit_transform(df['transformed_text']).toarray()
```

```
X.shape
```

```
(5169, 6708)
```

```
y = df['target'].values
```

```
y.shape
```

```
(5169,)
```

Dividindo em treino, teste:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 2)
```

Vamos testar diferentes modelos e investigar qual performa melhor:

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score
```

Esses três modelos do Naive Bayes — GaussianNB, MultinomialNB e BernoulliNB — foram escolhidos porque são algoritmos eficientes para problemas de classificação de texto, como identificar se uma mensagem é spam ou ham.

```
gnb = GaussianNB()
mnb = MultinomialNB()
bnb = BernoulliNB()
```

```
# Primeiro modelo:
gnb.fit(X_train, y_train)
y_pred1 = gnb.predict(X_test)
print(accuracy_score(y_test, y_pred1))
print(confusion_matrix(y_test, y_pred1))
print(precision_score(y_test, y_pred1))
```

```
0.8800773694390716
[[792 104]
 [ 20 118]]
0.5315315315315315
```

```
# Segundo modelo:
mnb.fit(X_train, y_train)
y_pred2 = mnb.predict(X_test)
print(accuracy_score(y_test, y_pred2))
print(confusion_matrix(y_test, y_pred2))
print(precision_score(y_test, y_pred2))
```

```
0.9642166344294004
[[871  25]
 [ 12 126]]
0.8344370860927153
```

```
# Terceiro modelo:

bnb.fit(X_train, y_train)
y_pred3 = bnb.predict(X_test)
print(accuracy_score(y_test, y_pred3))
print(confusion_matrix(y_test, y_pred3))
print(precision_score(y_test, y_pred3))
```

```
0.9700193423597679
[[893   3]
 [ 28 110]]
0.9734513274336283
```

Vamos tentar outra abordagem para melhorar essas métricas. O modelo Multinomial Naive Bayes (MNB) está sendo usado em conjunto com a técnica TF-IDF (Term Frequency - Inverse Document Frequency) para transformar o texto em números antes de ser passado para o classificador.

```
from sklearn.feature_extraction.text import CountVectorizer,TfidfVectorizer
tfidf = TfidfVectorizer()
```

```
X = tfidf.fit_transform(df['transformed_text']).toarray()
```

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=2)
```

```
from sklearn.naive_bayes import GaussianNB,MultinomialNB,BernoulliNB
from sklearn.metrics import accuracy_score,confusion_matrix,precision_score
```

```
gnb = GaussianNB()
mnb = MultinomialNB()
bnb = BernoulliNB()
```

```
gnb.fit(X_train,y_train)
y_pred1 = gnb.predict(X_test)
print(accuracy_score(y_test,y_pred1))
print(confusion_matrix(y_test,y_pred1))
print(precision_score(y_test,y_pred1))
```

```
0.8762088974854932
[[793 103]
 [ 25 113]]
0.5231481481481481
```

```
mnb.fit(X_train,y_train)
y_pred2 = mnb.predict(X_test)
print(accuracy_score(y_test,y_pred2))
print(confusion_matrix(y_test,y_pred2))
print(precision_score(y_test,y_pred2))
```

```
0.9593810444874274
[[896   0]
 [ 42  96]]
```

```
1.0
```

```
bnb.fit(X_train,y_train)
y_pred3 = bnb.predict(X_test)
print(accuracy_score(y_test,y_pred3))
print(confusion_matrix(y_test,y_pred3))
print(precision_score(y_test,y_pred3))
```

```
0.9700193423597679
[[893   3]
 [ 28 110]]
0.9734513274336283
```

Vimos que o melhor resultado foi atribuido ao uso do tfidf junto ao MNB! Vamos conduzir testes mais gerais para saber se iremos seguir assim!

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
```

```
svc = SVC(kernel='sigmoid', gamma=1.0)
knc = KNeighborsClassifier()
mnb = MultinomialNB()
dtc = DecisionTreeClassifier(max_depth=5)
lrc = LogisticRegression(solver='liblinear', penalty='l1')
rfc = RandomForestClassifier(n_estimators=50, random_state=2)
abc = AdaBoostClassifier(n_estimators=50, random_state=2)
bc = BaggingClassifier(n_estimators=50, random_state=2)
etc = ExtraTreesClassifier(n_estimators=50, random_state=2)
gbdt = GradientBoostingClassifier(n_estimators=50,random_state=2)
xgb = XGBClassifier(n_estimators=50,random_state=2)
```

```
clfs = {
    'SVC' : svc,
    'KN' : knc,
    'NB': mnb,
    'DT': dtc,
    'LR': lrc,
    'RF': rfc,
    'AdaBoost': abc,
    'BgC': bc,
    'ETC': etc,
    'GBDT':gbdt,
    'xgb':xgb
}
```

```
def train_classifier(clf,X_train,y_train,X_test,y_test):
    clf.fit(X_train,y_train)
    y_pred = clf.predict(X_test)
    accuracy = accuracy_score(y_test,y_pred)
    precision = precision_score(y_test,y_pred)

    return accuracy,precision
```

```
train_classifier(svc,X_train,y_train,X_test,y_test)
```

```
(0.9729206963249516, 0.9741379310344828)
```

```
accuracy_scores = []
precision_scores = []

for name,clf in clfs.items():

    current_accuracy,current_precision = train_classifier(clf, X_train,y_train,X_test,y_test)

    print("For ",name)
    print("Accuracy - ",current_accuracy)
    print("Precision - ",current_precision)
```

```
    accuracy_scores.append(current_accuracy)
    precision_scores.append(current_precision)
```

```
For  SVC
Accuracy -  0.9729206963249516
Precision -  0.9741379310344828
For  KN
Accuracy -  0.9003868471953579
Precision -  1.0
For  NB
Accuracy -  0.9593810444874274
Precision -  1.0
For  DT
Accuracy -  0.9361702127659575
Precision -  0.8461538461538461
For  LR
Accuracy -  0.9516441005802708
Precision -  0.94
For  RF
Accuracy -  0.971953578336557
Precision -  1.0
For  AdaBoost
Accuracy -  0.9245647969052224
Precision -  0.8409090909090909
For  BgC
Accuracy -  0.9584139264990329
Precision -  0.8625954198473282
For  ETC
Accuracy -  0.9729206963249516
Precision -  0.9824561403508771
For  GBDT
Accuracy -  0.9526112185686654
Precision -  0.9238095238095239
For  xgb
Accuracy -  0.9729206963249516
Precision -  0.9435483870967742
```

```
performance_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy':accuracy_scores,'Precision':precision_scores}).sort_values('Precision
performance_df
```

	Algorithm	Accuracy	Precision
1	KN	0.900387	1.000000
2	NB	0.959381	1.000000
5	RF	0.971954	1.000000
8	ETC	0.972921	0.982456
0	SVC	0.972921	0.974138
10	xgb	0.972921	0.943548
4	LR	0.951644	0.940000
9	GBDT	0.952611	0.923810
7	BgC	0.958414	0.862595
3	DT	0.936170	0.846154
6	AdaBoost	0.924565	0.840909

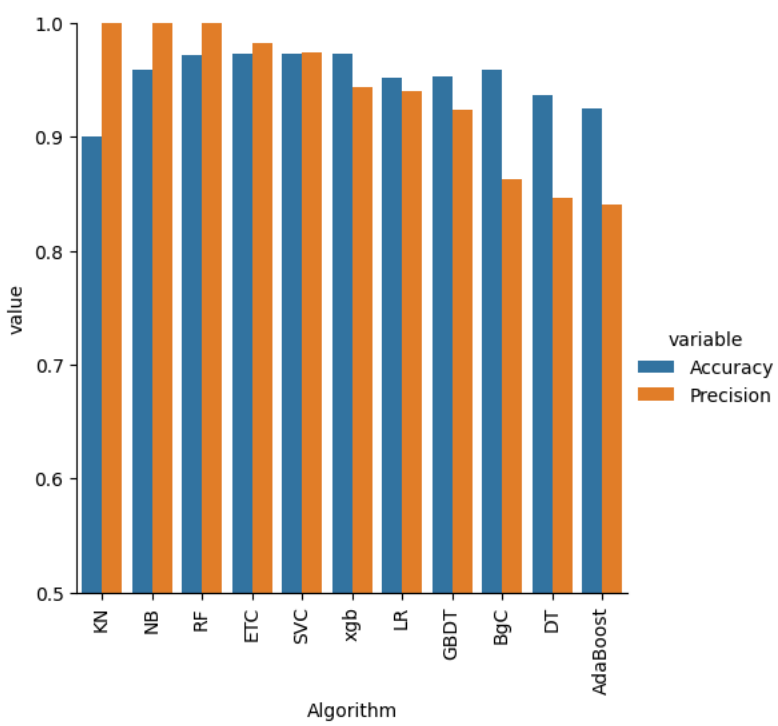
Próximas etapas: Gerar código com performance_df | Ver gráficos recomendados | New interactive sheet

```
performance_df1 = pd.melt(performance_df, id_vars = "Algorithm")
performance_df1
```

	Algorithm	variable	value
0	KN	Accuracy	0.900387
1	NB	Accuracy	0.959381
2	RF	Accuracy	0.971954
3	ETC	Accuracy	0.972921
4	SVC	Accuracy	0.972921
5	xgb	Accuracy	0.972921
6	LR	Accuracy	0.951644
7	GBDT	Accuracy	0.952611
8	BgC	Accuracy	0.958414
9	DT	Accuracy	0.936170
10	AdaBoost	Accuracy	0.924565
11	KN	Precision	1.000000
12	NB	Precision	1.000000
13	RF	Precision	1.000000
14	ETC	Precision	0.982456
15	SVC	Precision	0.974138
16	xgb	Precision	0.943548
17	LR	Precision	0.940000
18	GBDT	Precision	0.923810
19	BgC	Precision	0.862595
20	DT	Precision	0.846154
21	AdaBoost	Precision	0.840909

Próximas etapas: Gerar código com performance_df1 | Ver gráficos recomendados | New interactive sheet

```
sns.catplot(x = 'Algorithm', y='value',
               hue = 'variable',data=performance_df1, kind='bar',height=5)
plt.ylim(0.5,1.0)
plt.xticks(rotation='vertical')
plt.show()
```



O gráfico compara os algoritmos de Machine Learning em termos de Accuracy (acurácia) e Precision (precisão para a classe positiva). O modelo K-Neighbors (KN) apresentou o melhor desempenho, com alta acurácia e precisão próximas de 1.0, sendo o mais indicado. O Naive

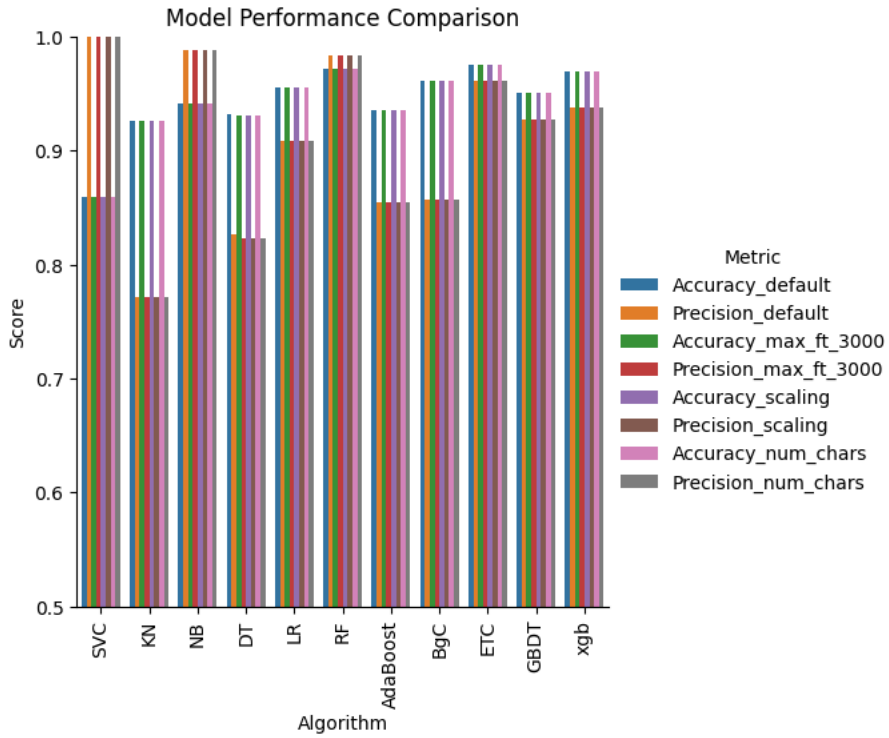
Bayes (NB) também se destacou, com resultados consistentes em ambas as métricas. Modelos como Random Forest (RF), Extra Trees (ETC) e Support Vector Classifier (SVC) tiveram bom desempenho. Por outro lado, Decision Tree (DT) e AdaBoost apresentaram menor precisão, sendo menos eficazes para o problema.

8. Model improving:

Mudando os parâmetros de max_features do TfIdf:

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, BaggingClassifier, ExtraTreesClassifier, GradientBoostingClassif
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, precision_score
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Data Preparation
cv = CountVectorizer()
tfidf = TfidfVectorizer(max_features=3000)
X = tfidf.fit_transform(df['transformed_text']).toarray()
y = df['target'].values

# Example Scaling (commented out for now)
# from sklearn.preprocessing import MinMaxScaler
# scaler = MinMaxScaler()
# X = scaler.fit_transform(X)

# Appending num_character column to X
X = np.hstack((X, df['num_characters'].values.reshape(-1, 1)))

# Splitting the data
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Model Definitions
clfs = {
    'SVC': SVC(kernel='sigmoid', gamma=1.0),
    'KN': KNeighborsClassifier(),
    'NB': MultinomialNB(),
    'DT': DecisionTreeClassifier(max_depth=5),
    'LR': LogisticRegression(solver='liblinear', penalty='l1'),
    'RF': RandomForestClassifier(n_estimators=50, random_state=2),
    'AdaBoost': AdaBoostClassifier(n_estimators=50, random_state=2),
    'BgC': BaggingClassifier(n_estimators=50, random_state=2),
    'ETC': ExtraTreesClassifier(n_estimators=50, random_state=2),
    'GBDT': GradientBoostingClassifier(n_estimators=50, random_state=2),
    'xgb': XGBClassifier(n_estimators=50, random_state=2)
}

# Train Classifier Function
def train_classifier(clf, X_train, y_train, X_test, y_test):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=1)
    return accuracy, precision

# Placeholder for DataFrame
performance_df = pd.DataFrame({'Algorithm': list(clfs.keys())})

# Loop through each model and calculate metrics at different stages
for step, description in zip(['default', 'max_ft_3000', 'scaling', 'num_chars'], ["Default", "Max Features 3000", "Scaling Applied", "Num
    accuracy_scores = []
    precision_scores = []

    for name, clf in clfs.items():
        acc, prec = train_classifier(clf, X_train, y_train, X_test, y_test)
        accuracy_scores.append(acc)
        precision_scores.append(prec)

    performance_df[f'Accuracy_{step}'] = accuracy_scores
    performance_df[f'Precision_{step}'] = precision_scores
```

```
# Plotting the final performance
performance_df1 = pd.melt(performance_df, id_vars="Algorithm", var_name="Metric", value_name="Score")
sns.catplot(x='Algorithm', y='Score', hue='Metric', data=performance_df1, kind='bar', height=5)
plt.ylim(0.5, 1.0)
plt.xticks(rotation='vertical')
plt.title("Model Performance Comparison")
plt.show()

# Display the final DataFrame
print(performance_df)
```

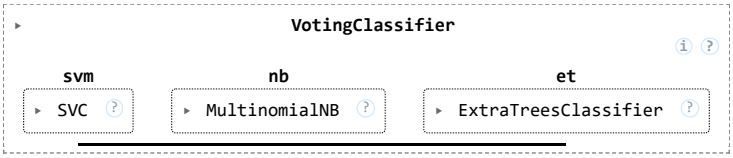



```
    Algorithm  Accuracy_default  Precision_default  Accuracy_max_ft_3000  \
0         SVC          0.859768           1.000000              0.859768
1          KN          0.926499           0.771654              0.926499
2          NB          0.941973           0.988506              0.941973
3          DT          0.932302           0.826087              0.930368
4          LR          0.955513           0.909091              0.955513
5          RF          0.971954           0.983333              0.971954
6    AdaBoost          0.935203           0.854545              0.935203
7         BgC          0.961315           0.857143              0.961315
8         ETC          0.974855           0.961240              0.974855
9        GBDT          0.950677           0.927273              0.950677
10        xgb          0.969052           0.937984              0.969052

    Precision_max_ft_3000  Accuracy_scaling  Precision_scaling  \
0                1.000000          0.859768           1.000000
1                0.771654          0.926499           0.771654
2                0.988506          0.941973           0.988506
3                0.823009          0.930368           0.823009
4                0.909091          0.955513           0.909091
5                0.983333          0.971954           0.983333
6                0.854545          0.935203           0.854545
7                0.857143          0.961315           0.857143
8                0.961240          0.974855           0.961240
9                0.927273          0.950677           0.927273
10               0.937984          0.969052           0.937984

    Accuracy_num_chars  Precision_num_chars
0             0.859768             1.000000
1             0.926499             0.771654
2             0.941973             0.988506
3             0.930368             0.823009
4             0.955513             0.909091
5             0.971954             0.983333
6             0.935203             0.854545
7             0.961315             0.857143
8             0.974855             0.961240
9             0.950677             0.927273
10            0.969052             0.937984
```

O gráfico compara o desempenho de vários algoritmos de classificação usando diferentes métricas, como Accuracy e Precision. Os melhores modelos são Random Forest (RF), Extra Trees Classifier (ETC) e XGBoost (xgb), com accuracy acima de 97% e alta precisão. O Naive Bayes

(NB) também apresenta boa performance, especialmente em precisão (98.85%). Algoritmos como Decision Tree (DT) e AdaBoost tiveram resultados razoáveis, mas com menor precisão. Modelos como K-Neighbors (KN) e Logistic Regression (LR) também performaram bem, mas com precisão um pouco menor. O SVC apresentou a maior precisão padrão, mas menor acurácia geral.

```
# Cirando um modelo com os melhores desempenhos:

svc = SVC(kernel='sigmoid', gamma=1.0,probability=True)
mnb = MultinomialNB()
etc = ExtraTreesClassifier(n_estimators=50, random_state=2)

from sklearn.ensemble import VotingClassifier


voting = VotingClassifier(estimators=[('svm', svc), ('nb', mnb), ('et', etc)],voting='soft')



voting.fit(X_train,y_train)
```




```
y_pred = voting.predict(X_test)
print("Accuracy",accuracy_score(y_test,y_pred))
print("Precision",precision_score(y_test,y_pred))
```

```
Accuracy 0.9410058027079303
Precision 1.0
```